

Patterns

Anti-Patterns and Refactoring

YEGOR BUGAYENKO

Lecture #6 out of 16

90 minutes

All videos are in [this YouTube playlist](#).

All visual and text materials presented in this slidedeck are either originally made by the author or taken from public Internet sources, such as website. Copyright belongs to their respected authors.



“Experienced designers evidently know something inexperienced ones don’t. What is it? One thing expert designers know not to do is solve every problem from first principles. Rather, they reuse solutions that have worked for them in the past. When they find a good solution, they use it again and again. Such experience is part of what makes them experts.”

— Design Patterns: Elements of Reusable Object-Oriented Software, Erich Gamma et al.



Some Anti-Patterns

Anti-OOP Patterns

Some Refactorings

Books, Venues, Call-to-Action

Chapter #1:

Some Anti-Patterns

[GOTO Numbers God Spaghetti Lasagna]

GOTO

```
void foo(int a) {%  
    if (a % 2 == 0) {%  
        printf("Even!");  
        goto exit;  
    }  
    printf("Odd!");  
exit:  
}  
  
void foo(int a) {%  
    if (a % 2 == 0) {%  
        printf("Even!");  
    } else {%  
        printf("Odd!");  
    }  
}
```

[GOTO [Numbers](#) God Spaghetti Lasagna]

Magic Numbers

```
def points
  File.readlines("/data/users.csv") # why here?
  .map { |t| t.split(',', 11) } # what is 11?
  .map { a[7].to_i } # why 7?
  .inject(&:+)
end
```

[GOTO [Numbers](#) God Spaghetti Lasagna]

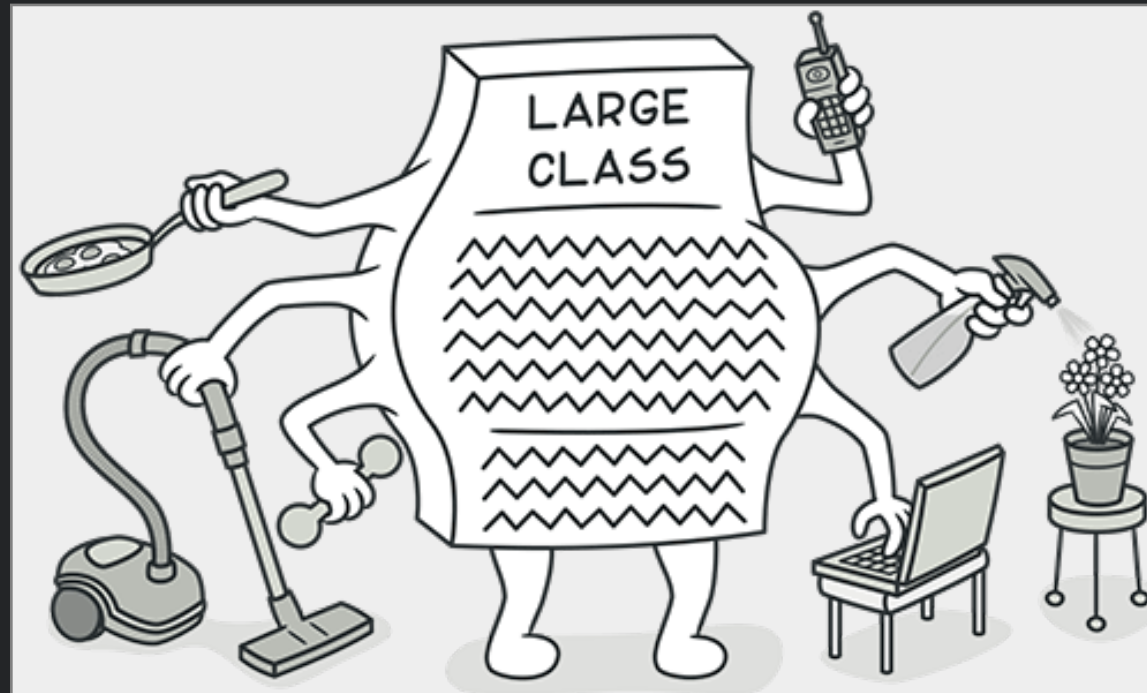
Magic Numbers ... Not!

```
def h2sec(h)
  return h * 60 * 60
end
```

```
def (h)
  seconds_in_minutes = 60
  minutes_in_hours = 60
  return h * seconds_in_minutes * minutes_in_hours
end
```

[GOTO Numbers God Spaghetti Lasagna]

God Class



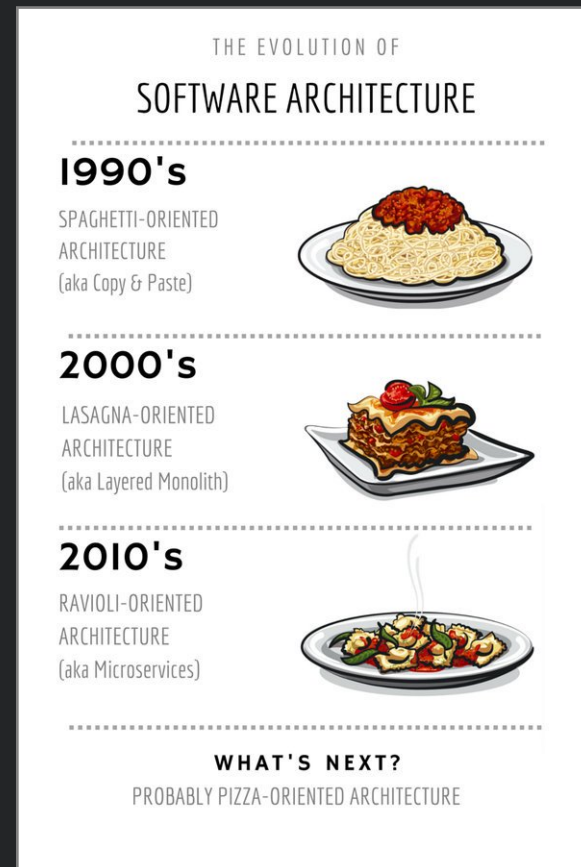
[GOTO Numbers God Spaghetti Lasagna]

Spaghetti Code

```
1 C      A weird program for calculating Pi written in Fortran.
2 C      From: Fink, D.G., Computers and the Human Mind, Anchor Books, 1966.
3
4      PROGRAM PI
5      DIMENSION TERM(100)
6      N=1
7      TERM(N)=((-1)**(N+1))*(4./(2.*N-1.))
8      N=N+1
9      IF (N-101) 3,6,6
10     N=1
11     SUM98 = SUM98+TERM(N)
12     WRITE(*,28) N, TERM(N)
13     N=N+1
14     IF (N-99) 7, 11, 11
15     SUM99=SUM98+TERM(N)
16     SUM100=SUM99+TERM(N+1)
17     IF (SUM98-3.141592) 14,23,23
18     IF (SUM99-3.141592) 23,23,15
19     IF (SUM100-3.141592) 16,23,23
20     AV89=(SUM98+SUM99)/2.
21     AV90=(SUM99+SUM100)/2.
22     COMANS=(AV89+AV90)/2.
23     IF (COMANS-3.1415920) 21,19,19
24     IF (COMANS-3.1415930) 20,21,21
25     WRITE(*,26)
26     GO TO 22
27     WRITE(*,27) COMANS
28     STOP
29     WRITE(*,25)
30     GO TO 22
31     FORMAT('ERROR IN MAGNITUDE OF SUM')
32     FORMAT('PROBLEM SOLVED')
33     FORMAT('PROBLEM UNSOLVED', F14.6)
34     FORMAT(I3, F14.6)
35     END
36
```

[GOTO Numbers God Spaghetti Lasagna]

Lasagna and Ravioli



[GOTO Numbers God Spaghetti Lasagna]

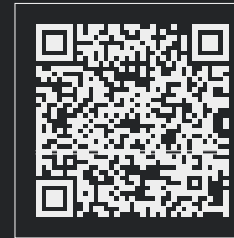
Data Transfer Object (DTO) Getters and Setters

// Getters and Setters: WRONG!

```
Dog dog = new Dog();  
dog.setWeight("23kg");  
w = dog.getWeight();
```

// Smart objects: RIGHT!

```
Dog dog = new Dog("23kg");  
int w = dog.weight();
```



<https://www.yegor256.com/2014/09/16/getters-and-setters-are-evil.html>
→

Chapter #2:

Anti-Al Patterns

[[DTO](#) Utility Singleton ORM]

Demanding 100% Accuracy

Demanding 100% Accuracy

Expecting LLM/model to be perfect (especially in legal, medical, finance).
Why bad: LLMs are probabilistic → impossible. Leads to impossible requirements, blame game.

Prompt Engineering as Architecture

Cramming all business logic into giant prompts (no real code structure).
Why bad: Hard to debug, expensive (long prompts = high tokens), breaks easily.

Hype Chasing / Model of the Week

Switching models constantly (Llama 3 → Grok-2 → Qwen 3 → next...).

[DTO Utility Singleton [ORM](#)]

No Evaluation / Blind Deployment

Train → deploy without proper eval, A/B test, monitoring.

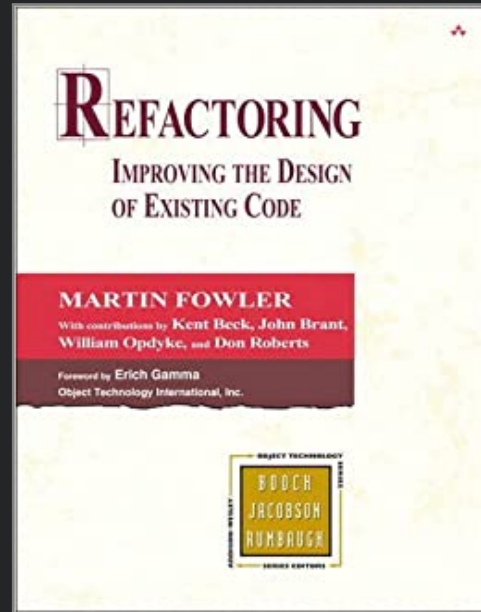
[DTO Utility Singleton ORM]

The Golden LLM (LLM version of Golden Hammer)

Using LLM for everything (even when cheaper/better options exist).

Chapter #3:

Some Refactorings



“Whenever I do refactoring, the first step is always the same. I need to build a solid set of tests for that section of code. The tests are essential because even though I follow refactorings structured to avoid most of the opportunities for introducing bugs, I’m still human and still make mistakes. Thus I need solid tests.”

— Refactoring: Improving the Design of Existing Code, Martin Fowler

Extract Method

```
def root(a, b, c)
  d = Math.sqrt(b * b - 4 * a * c)
  r1 = (-b + d) / (2 * a)
  r2 = (-b - d) / (2 * a)
  [r1, r2]
end
```

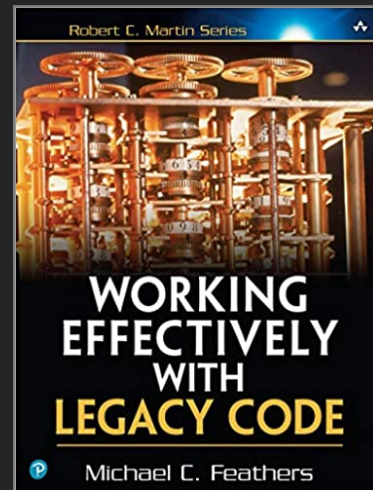
```
def root(a, b, c)
  d = Math.sqrt(b * b - 4 * a * c)
  [r(a, b, d, 1), r(a, b, d, -1)]
end

def r(a, b, d, m)
  (-b + d * m) / (2 * a)
end
```

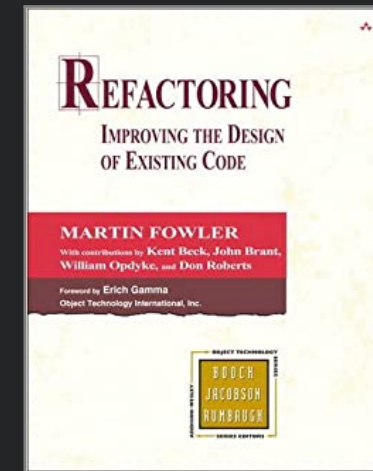
$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Chapter #4:

Books, Venues, Call-to-Action



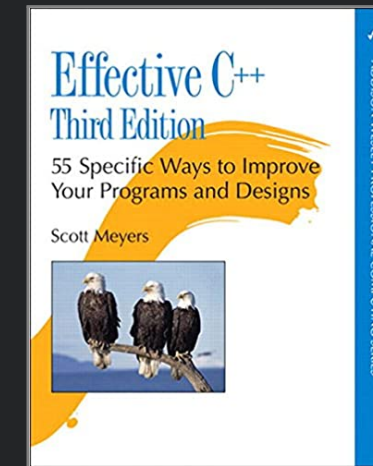
“Working Effectively with Legacy Code” by MICHAEL FEATHERS



“Refactoring: Improving the Design of Existing Code” by MARTIN FOWLER



“Design Patterns: Elements of Reusable Object-Oriented Software” by ERICH GAMMA ET AL.



“Effective C++: 55 Specific Ways to Improve Your Programs and Designs” by SCOTT MEYERS

Where to publish:

SPLASH: ACM SIGPLAN conference on Systems, Programming, Languages, and Applications

International Conference on Code Quality (ICCCQ),
in cooperation with ACM SIGPLAN/SIGSOFT and IEEE

Call to Action:

In your application demonstrate the usage of 4+ design patterns. Also, perform 4+ refactorings, each one in its own pull request.

Still unresolved issues:

- How to prove certain patterns are anti-patterns?
- How to find methods for automated refactoring?
- How to guarantee validity during refactoring?
- How to mine patterns from code?